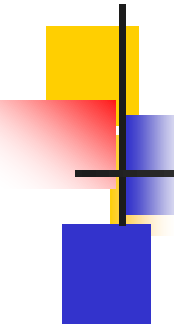


# Introduction to Support Vector Machines

---



# History of SVM

---

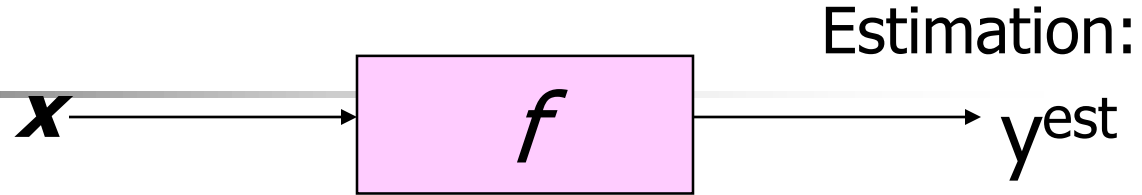
- SVM is related to statistical learning theory [3]
- SVM was first introduced in 1992 [1]
- SVM becomes popular because of its success in handwritten digit recognition
  - 1.1% test error rate for SVM. This is the same as the error rates of a carefully constructed neural network, LeNet 4.
    - See Section 5.11 in [2] or the discussion in [3] for details
- SVM is now regarded as an important example of “kernel methods”, one of the **key area in machine learning**
  - Note: the meaning of “kernel” is different from the “kernel” function for Parzen windows

[1] B.E. Boser *et al.* A Training Algorithm for Optimal Margin Classifiers. Proceedings of the Fifth Annual Workshop on Computational Learning Theory 5 144-152, Pittsburgh, 1992.

[2] L. Bottou *et al.* Comparison of classifier methods: a case study in handwritten digit recognition. Proceedings of the 12th IAPR International Conference on Pattern Recognition, vol. 2, pp. 77-82.

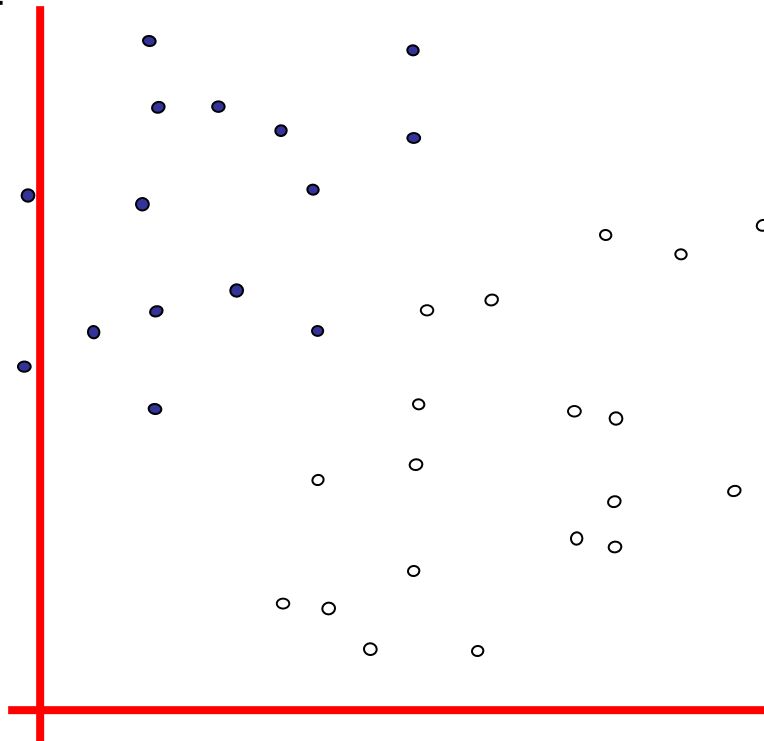
[3] V. Vapnik. The Nature of Statistical Learning Theory. 2<sup>nd</sup> edition, Springer, 1999.

# Linear Classifiers



$$f(\mathbf{x}, \mathbf{w}, b) = \text{sign}(\mathbf{w} \cdot \mathbf{x} - b)$$

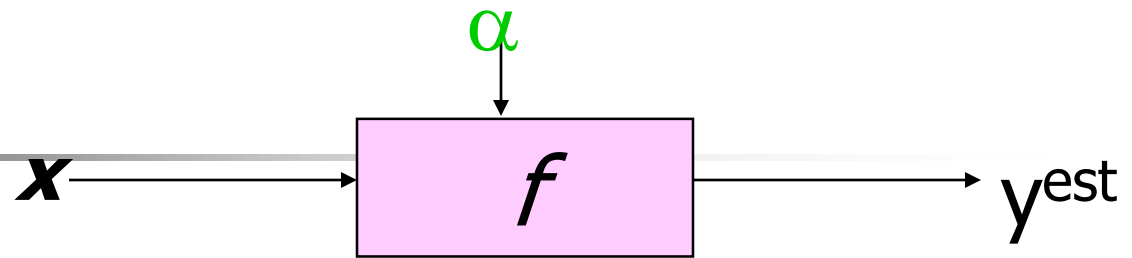
- denotes +1
- denotes -1



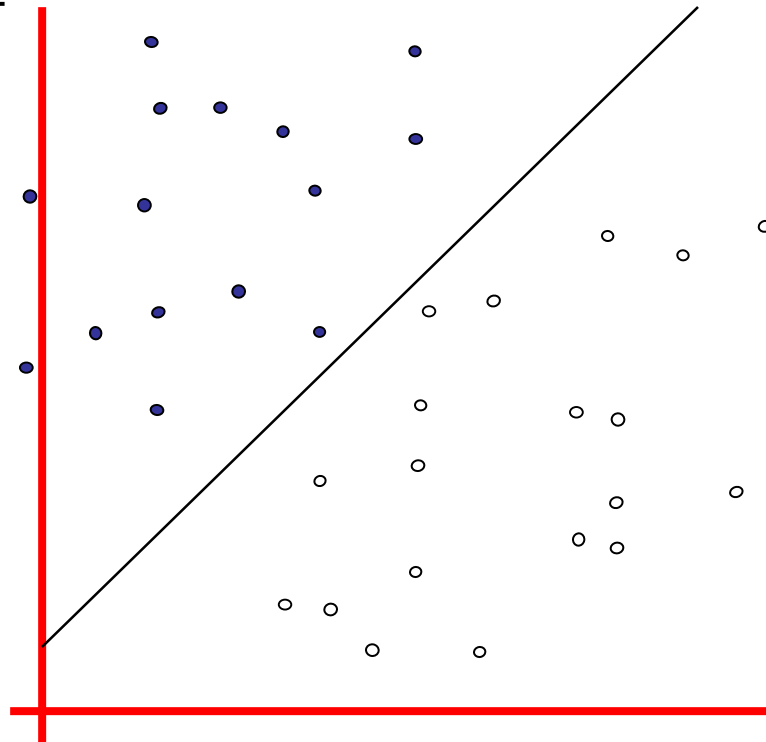
**w**: weight vector  
**x**: data vector

How would you  
classify this data?

# Linear Classifiers



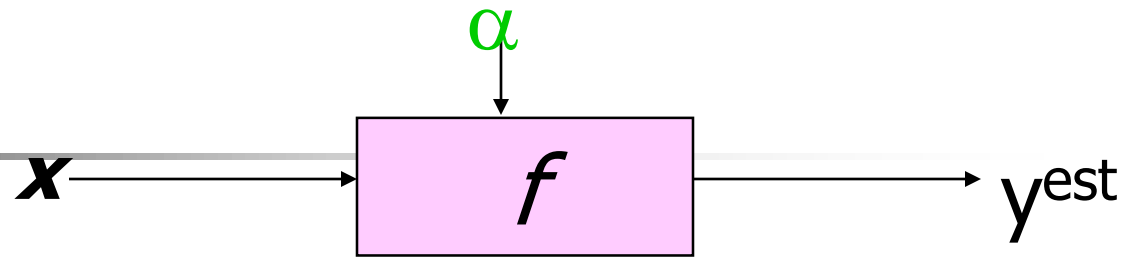
- denotes +1
- denotes -1



$$f(\mathbf{x}, \mathbf{w}, b) = \text{sign}(\mathbf{w} \cdot \mathbf{x} - b)$$

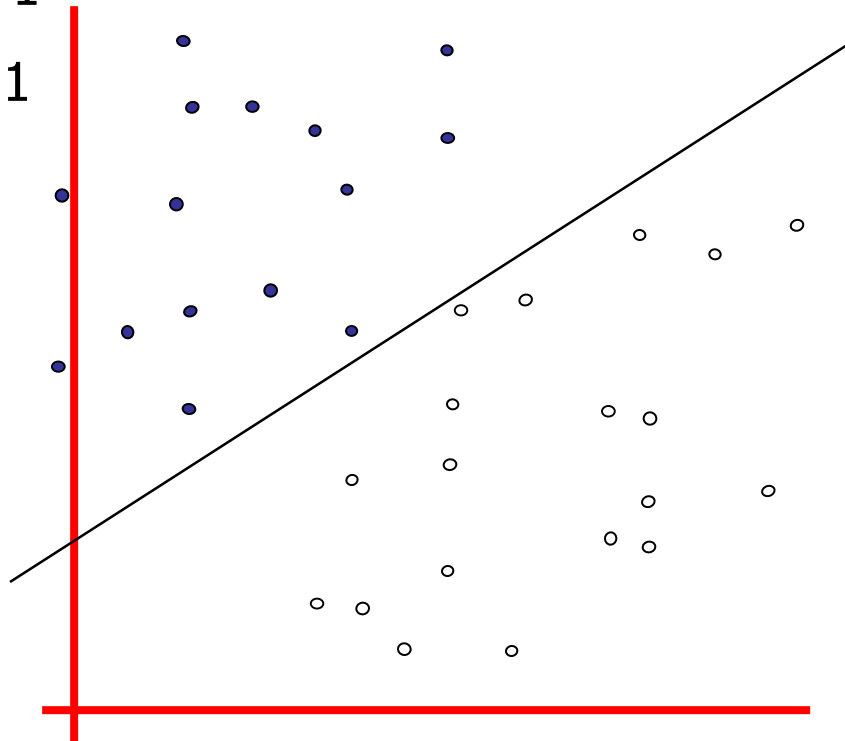
How would you classify this data?

# Linear Classifiers



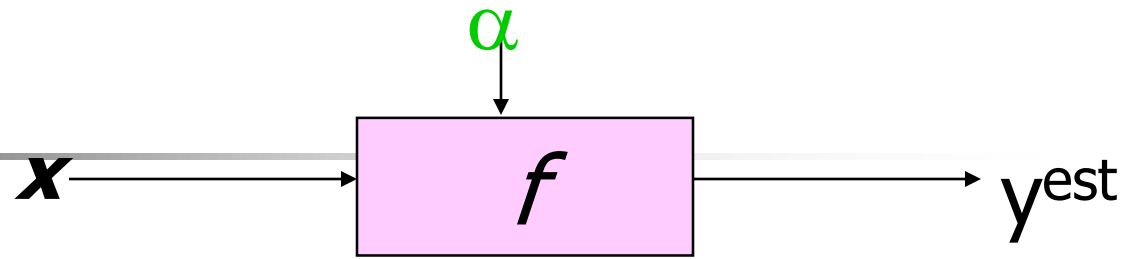
$$f(\mathbf{x}, \mathbf{w}, b) = \text{sign}(\mathbf{w} \cdot \mathbf{x} - b)$$

- denotes +1
- denotes -1

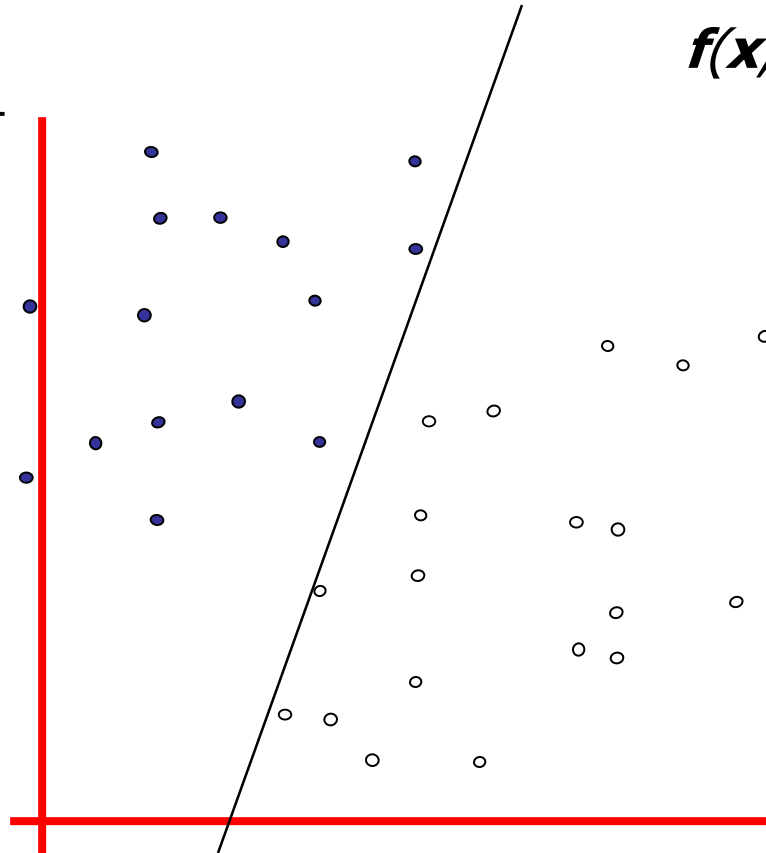


How would you classify this data?

# Linear Classifiers



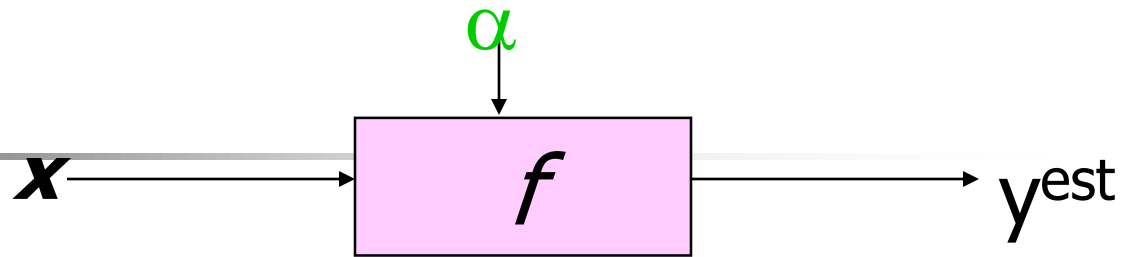
- denotes +1
- denotes -1



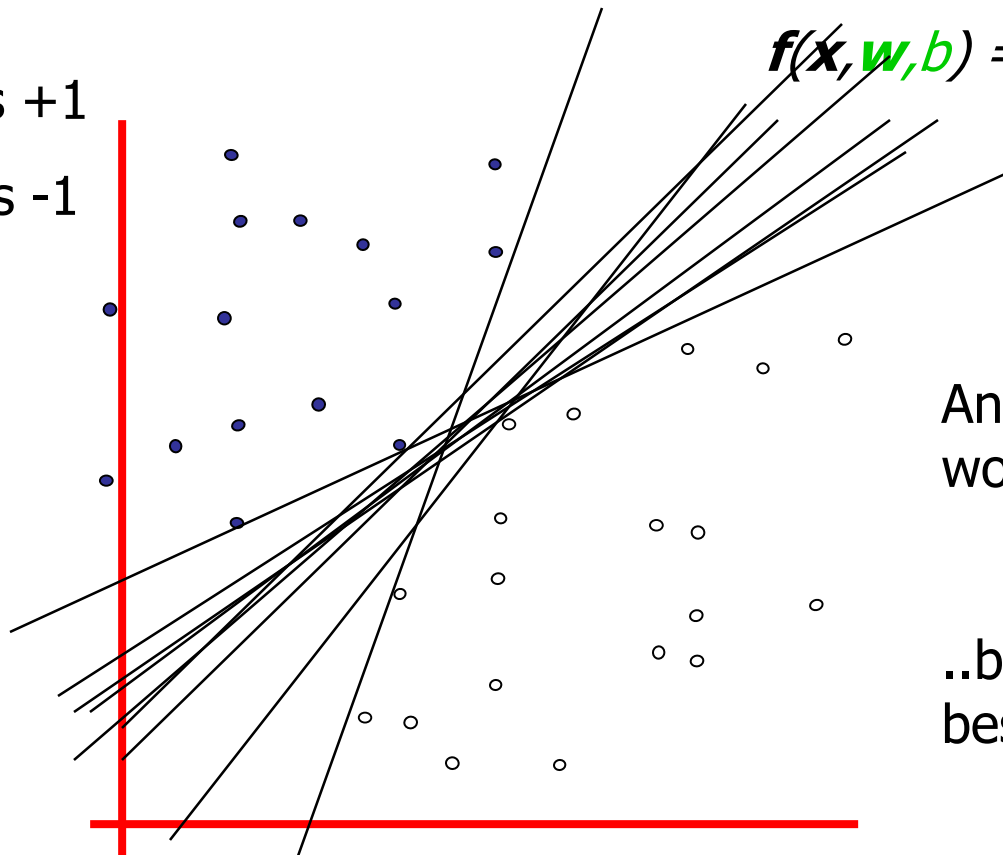
$$f(\mathbf{x}, \mathbf{w}, b) = \text{sign}(\mathbf{w} \cdot \mathbf{x} - b)$$

How would you classify this data?

# Linear Classifiers



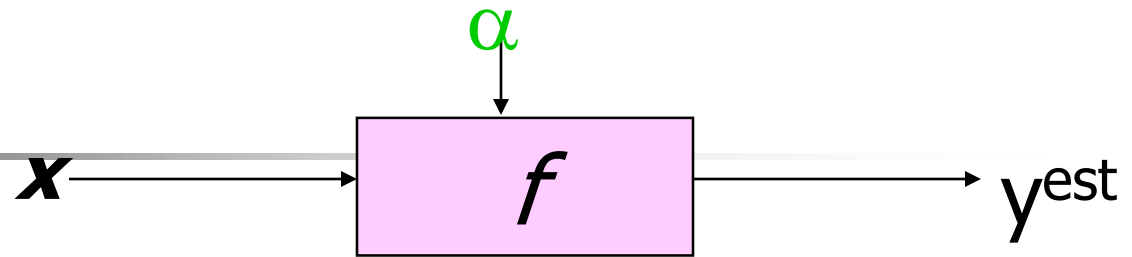
- denotes +1
- denotes -1



Any of these  
would be fine..

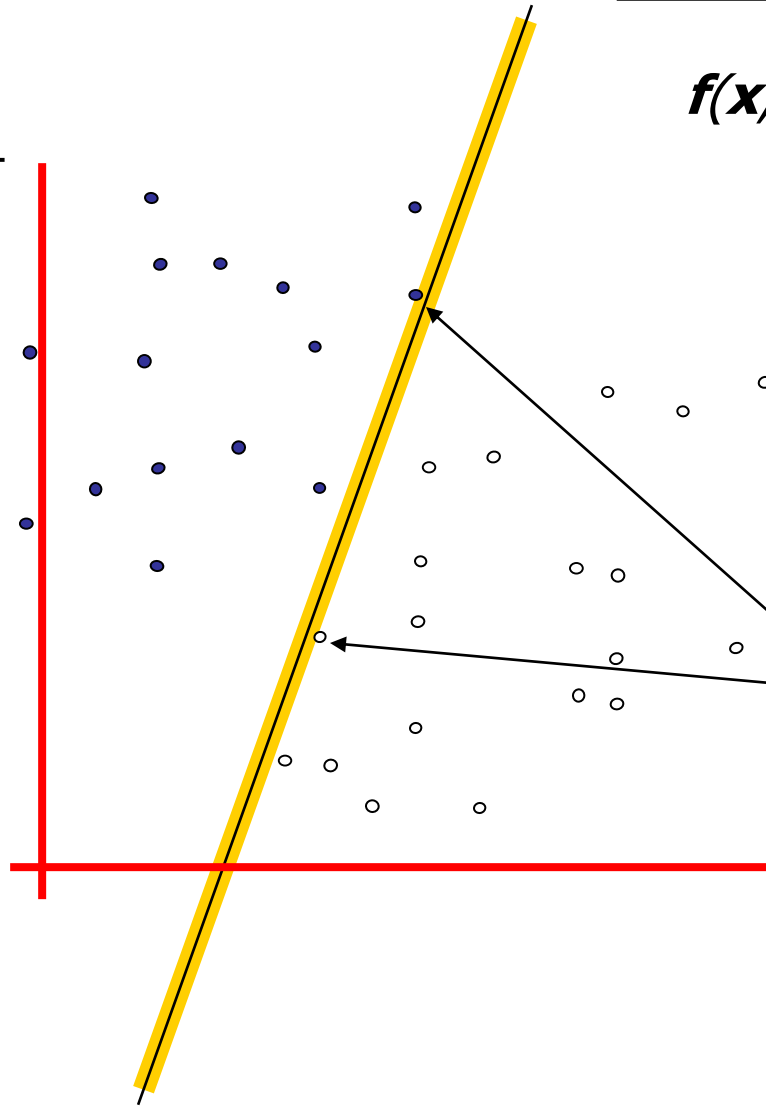
..but which is  
best?

# Classifier Margin



$$f(\mathbf{x}, \mathbf{w}, b) = \text{sign}(\mathbf{w} \cdot \mathbf{x} - b)$$

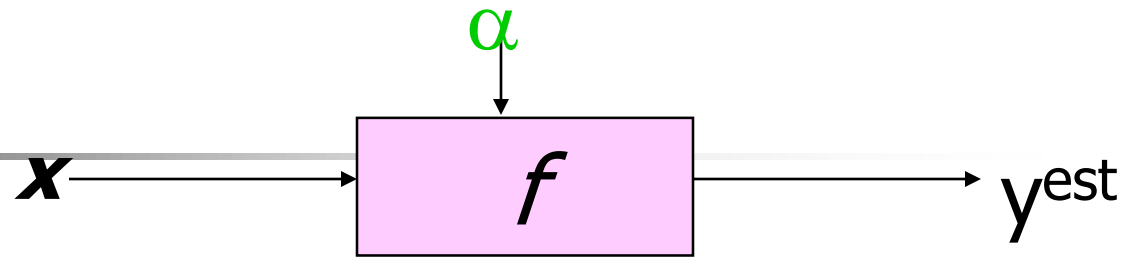
- denotes +1
- denotes -1



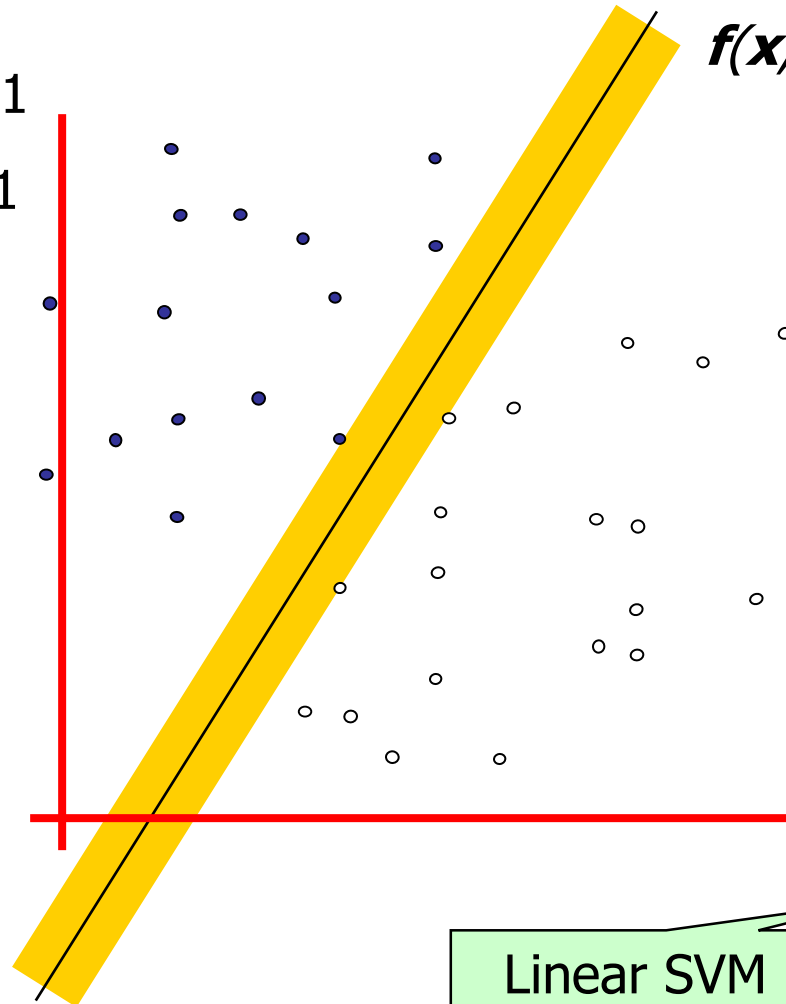
Define the **margin** of a linear classifier as the width that the boundary could be increased by **before hitting a datapoint.**



# Maximum Margin



- denotes +1
- denotes -1



$$f(\mathbf{x}, \mathbf{w}, b) = \text{sign}(\mathbf{w} \cdot \mathbf{x} - b)$$

The **maximum margin linear classifier** is the linear classifier with the, um, maximum margin.

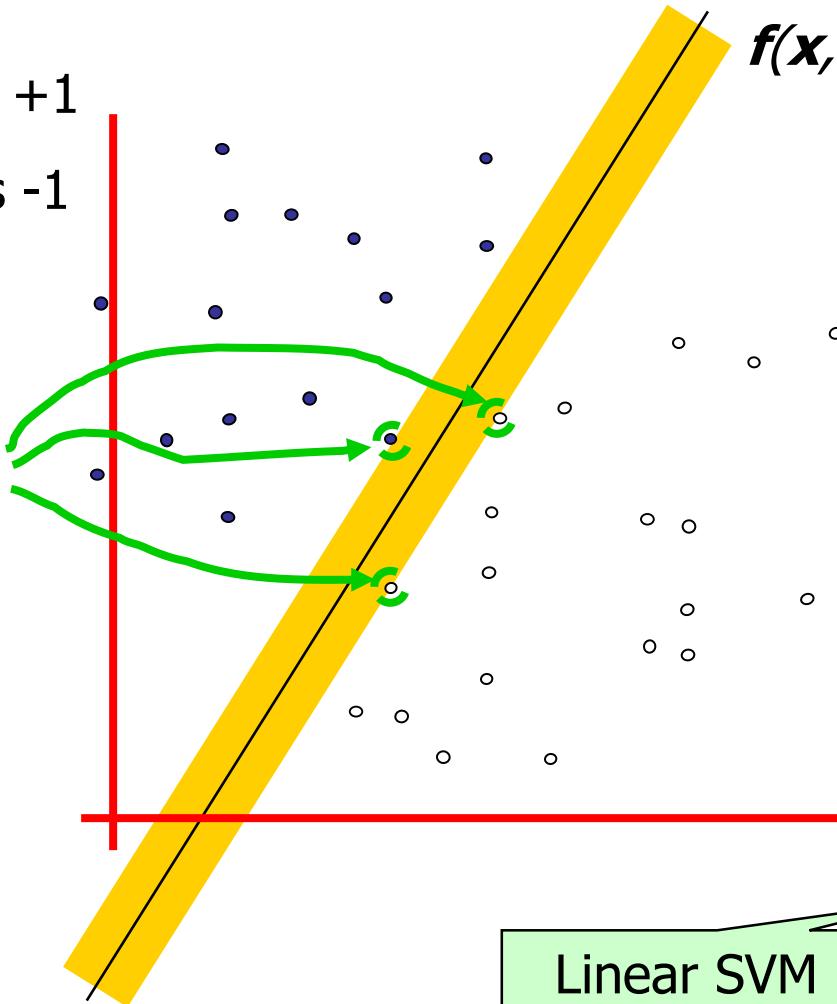
This is the simplest kind of SVM (Called an LSVM)

Linear SVM

# Maximum Margin

- denotes +1
- denotes -1

Support Vectors  
are those  
datapoints that  
the margin  
pushes up  
against



$$f(\mathbf{x}, \mathbf{w}, b) = \text{sign}(\mathbf{w} \cdot \mathbf{x} + b)$$

The **maximum margin linear classifier** is the linear classifier with the, um, maximum margin.

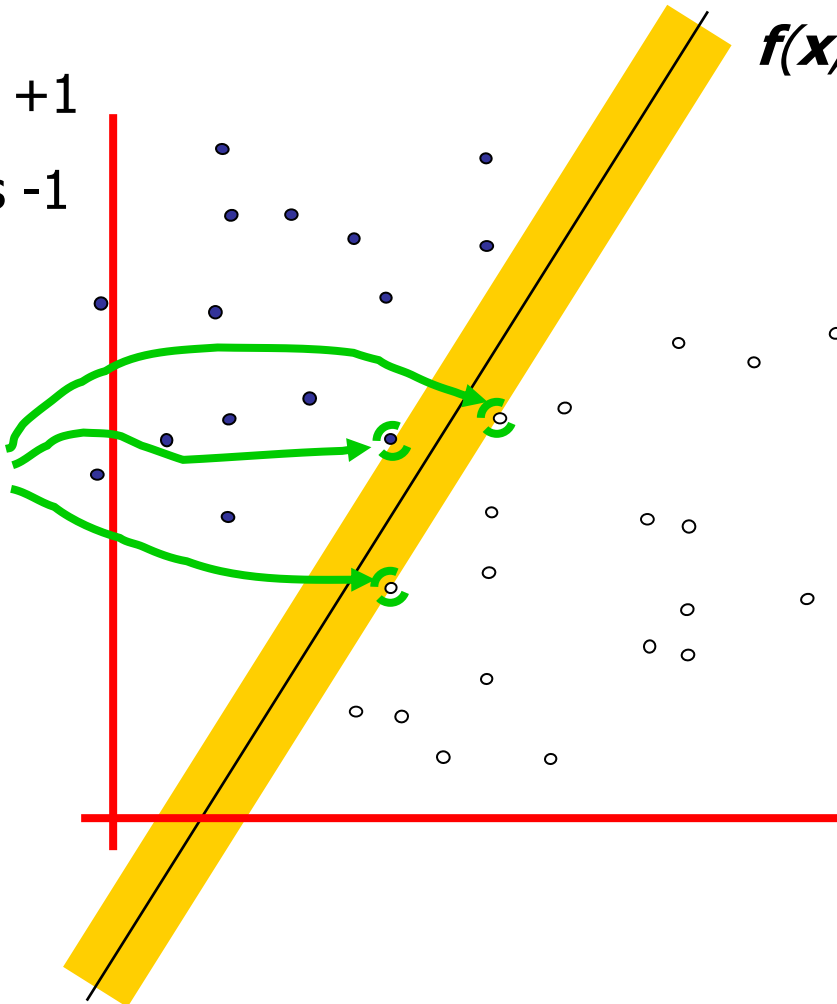
This is the simplest kind of SVM (Called an LSVM)

Linear SVM

# Why Maximum Margin?

- denotes +1
- denotes -1

Support Vectors  
are those  
datapoints that  
the margin  
pushes up  
against

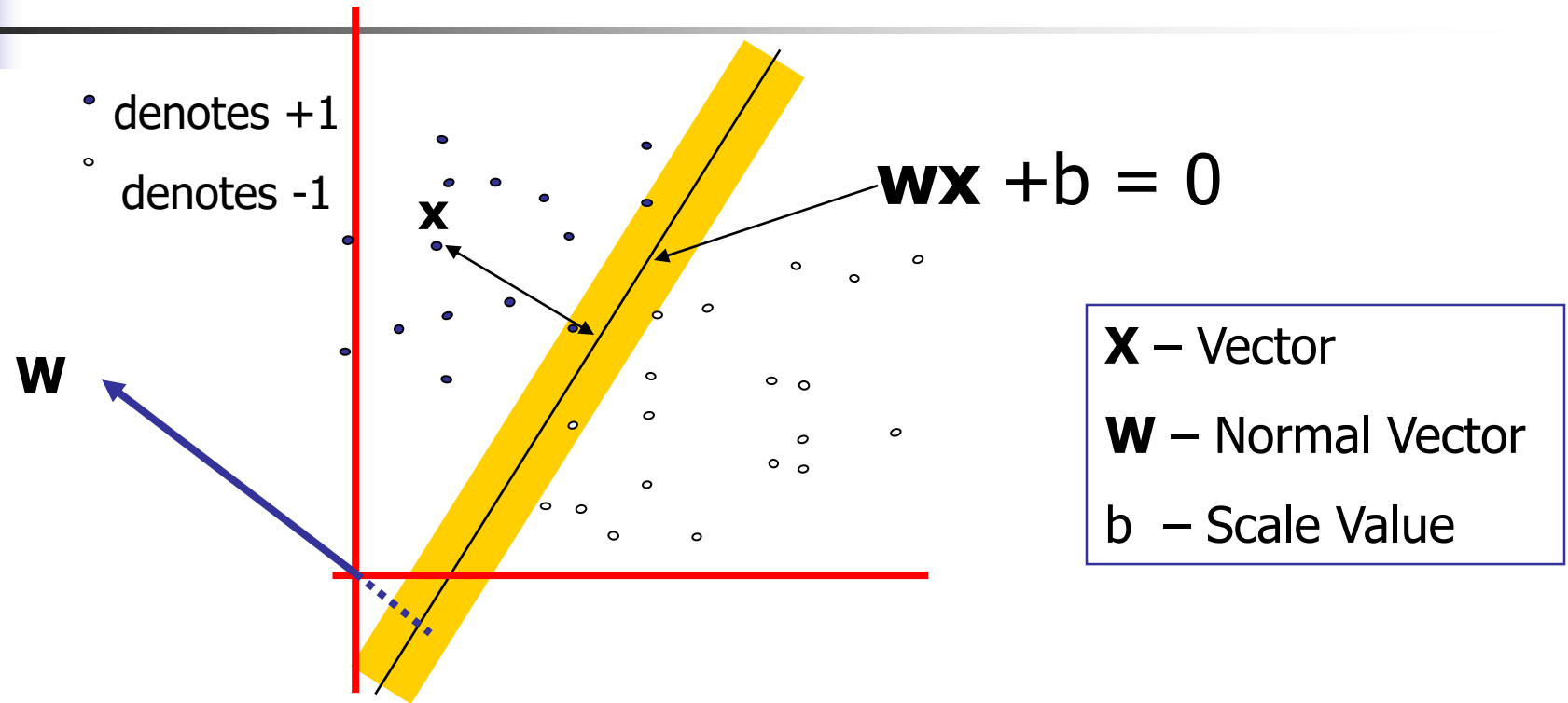


$$f(\mathbf{x}, \mathbf{w}, b) = \text{sign}(\mathbf{w} \cdot \mathbf{x} - b)$$

The **maximum margin linear classifier** is the linear classifier with the, um, maximum margin.

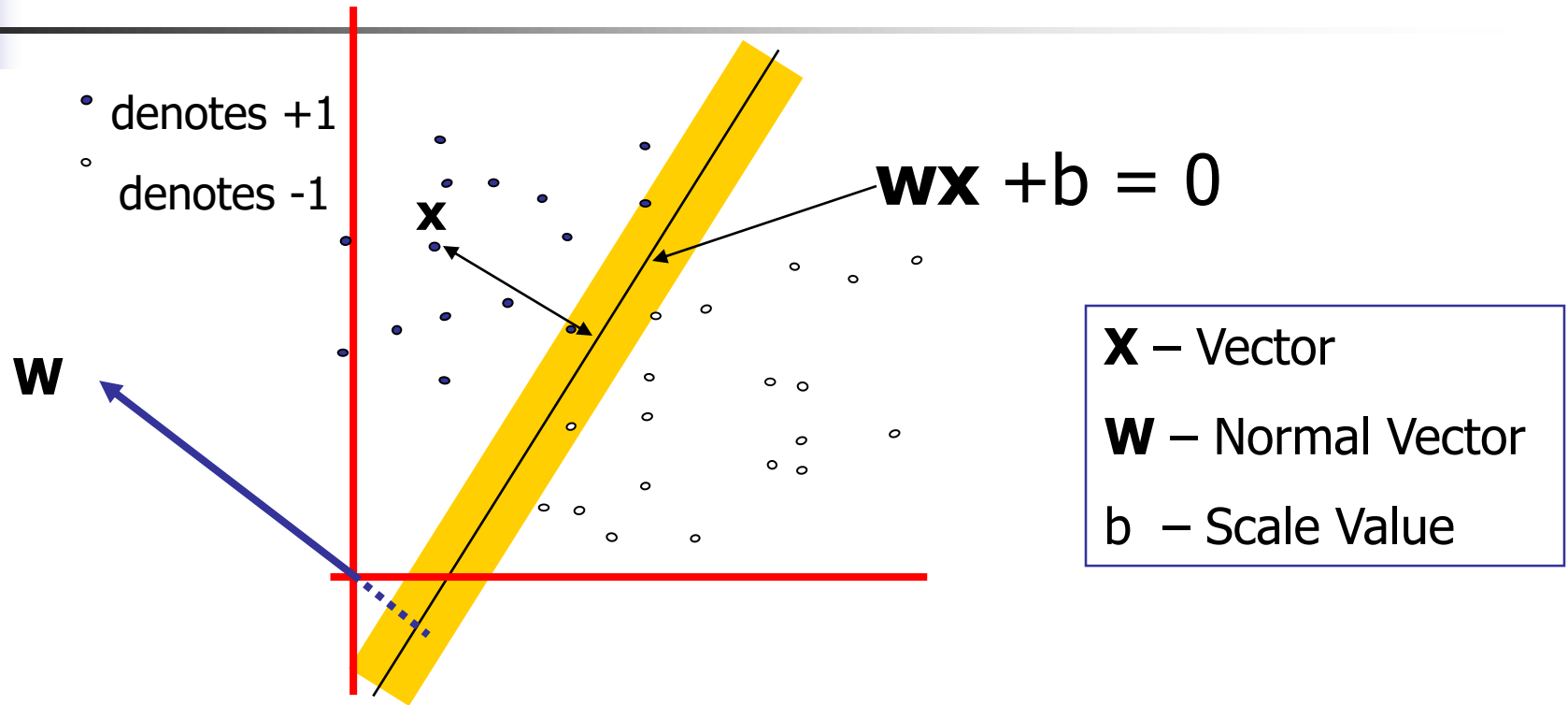
This is the simplest kind of SVM (Called an LSVM)

# How to calculate the distance from a point to a line?



- <http://mathworld.wolfram.com/Point-LineDistance2-Dimensional.html>
- In our case,  $w_1 * x_1 + w_2 * x_2 + b = 0$ ,
- thus,  $\mathbf{w} = (w_1, w_2)$ ,  $\mathbf{x} = (x_1, x_2)$

# Estimate the Margin

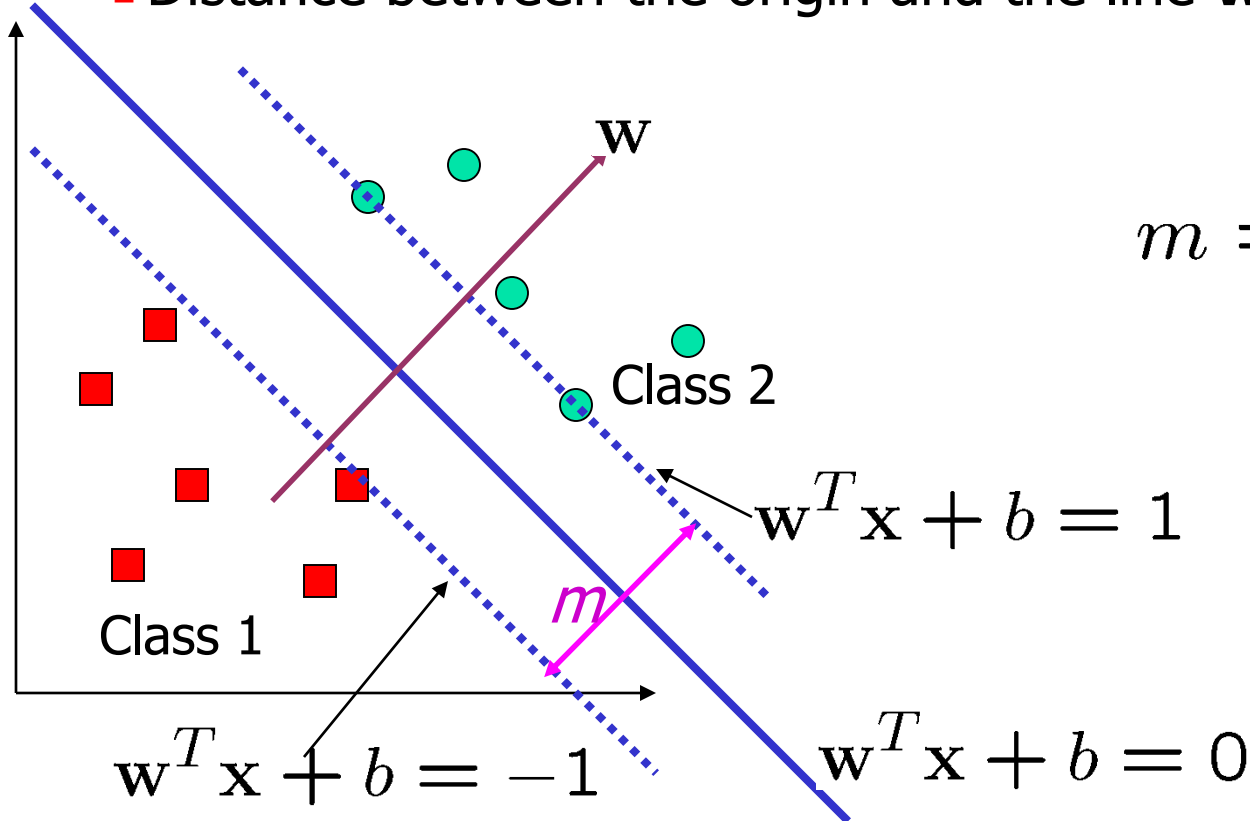


- What is the distance expression for a point  $\mathbf{x}$  to a line  $\mathbf{w}\mathbf{x} + b = 0$ ?

$$d(\mathbf{x}) = \frac{|\mathbf{x} \cdot \mathbf{w} + b|}{\sqrt{\|\mathbf{w}\|_2^2}} = \frac{|\mathbf{x} \cdot \mathbf{w} + b|}{\sqrt{\sum_{i=1}^d w_i^2}}$$

# Large-margin Decision Boundary

- The decision boundary should be as far away from the data of both classes as possible
  - We should maximize the margin,  $m$
  - Distance between the origin and the line  $\mathbf{w}^T \mathbf{x} = -b$  is  $b/||\mathbf{w}||$

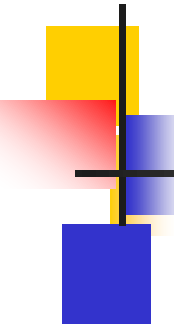


$$m = \frac{2}{||\mathbf{w}||}$$

# Finding the Decision Boundary

- Let  $\{x_1, \dots, x_n\}$  be our data set and let  $y_i \in \{1, -1\}$  be the class label of  $x_i$
- The decision boundary should classify all points correctly  
 $\Rightarrow y_i(w^T x_i + b) \geq 1, \quad \forall i$
- To see this: when  $y=-1$ , we wish  $(wx+b)<1$ , when  $y=1$ , we wish  $(wx+b)>1$ . For support vectors, we wish  $y(wx+b)=1$ .
- The decision boundary can be found by solving the following constrained optimization problem

$$\begin{aligned} & \text{Minimize } \frac{1}{2} \|w\|^2 \\ & \text{subject to } y_i(w^T x_i + b) \geq 1 \quad \forall i \end{aligned}$$



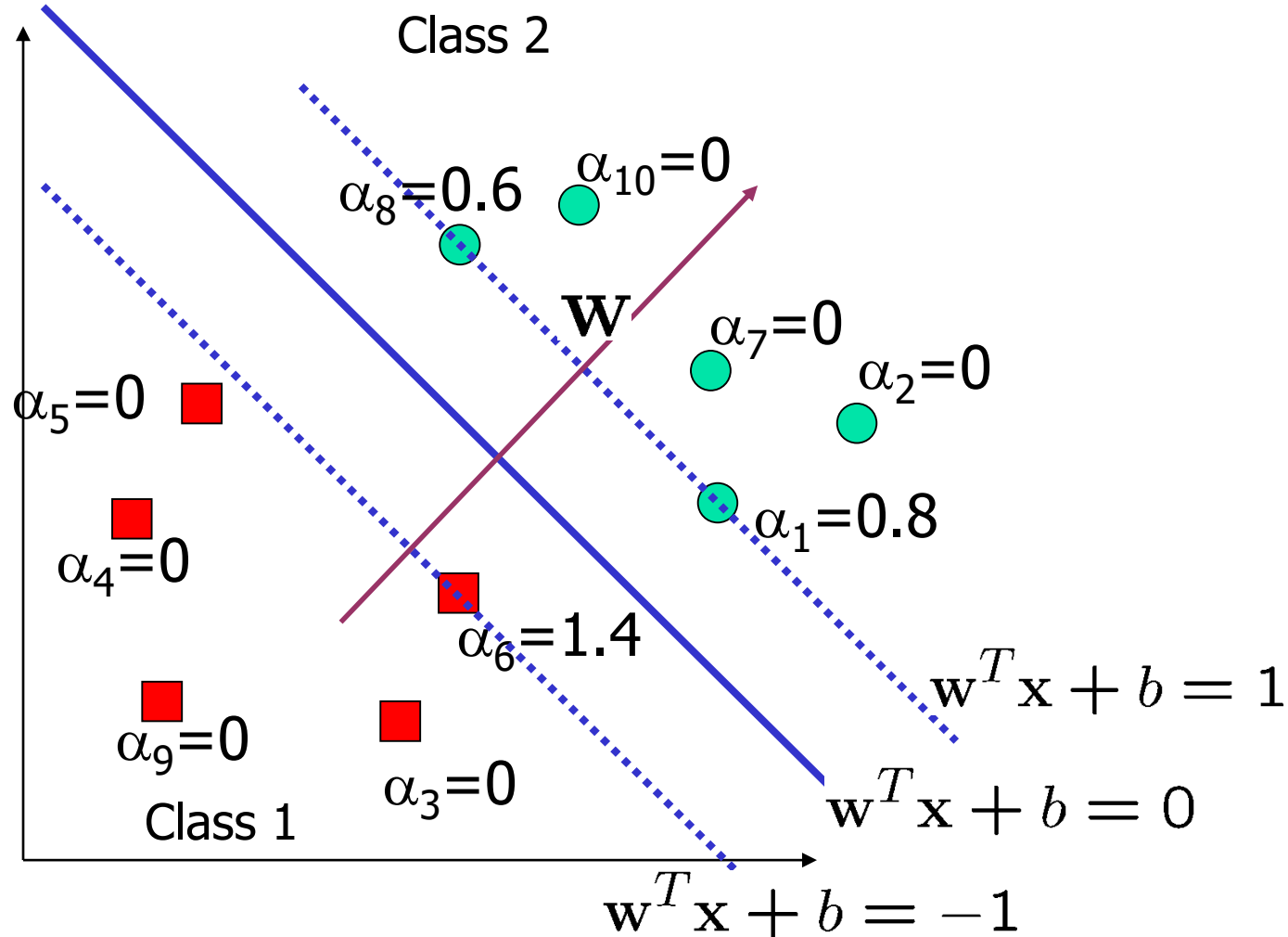
## Next step... Optional

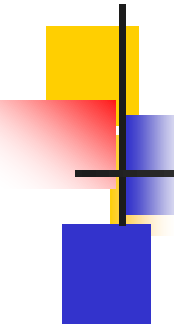
---

- Converting SVM to a form we can solve
  - Dual form
- Allowing a few errors
  - Soft margin
- Allowing nonlinear boundary
  - Kernel functions



# A Geometrical Interpretation



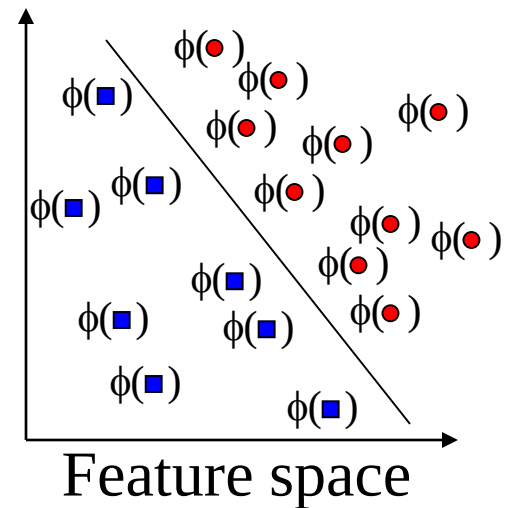
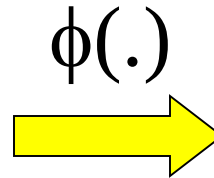
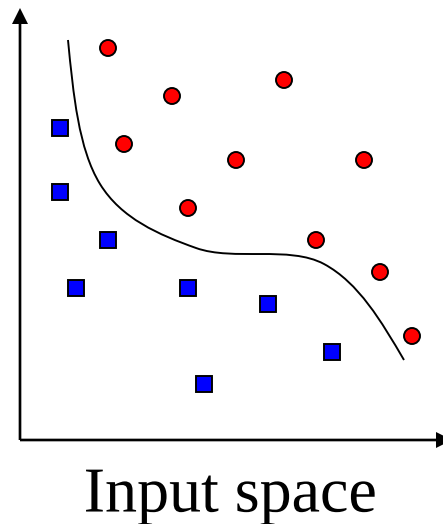


# Extension to Non-linear Decision Boundary

---

- So far, we have only considered large-margin classifier with a linear decision boundary
- How to generalize it to become nonlinear?
- Key idea: transform  $\mathbf{x}_i$  to a higher dimensional space to “make life easier”
  - Input space: the space the point  $\mathbf{x}_i$  are located
  - Feature space: the space of  $\phi(\mathbf{x}_i)$  after transformation

# Transforming the Data (c.f. DHS Ch. 5)



Note: feature space is of higher dimension than the input space in practice

- Computation in the feature space can be costly because it is high dimensional
  - The feature space is typically infinite-dimensional!
- The kernel trick comes to rescue

# The Kernel Trick

- Recall the SVM optimization problem

$$\begin{aligned} \max. \quad W(\alpha) &= \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1, j=1}^n \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j \\ \text{subject to } C &\geq \alpha_i \geq 0, \quad \sum_{i=1}^n \alpha_i y_i = 0 \end{aligned}$$

- The data points only appear as **inner product**
- As long as we can calculate the inner product in the feature space, we do not need the mapping explicitly
- Many common geometric operations (angles, distances) can be expressed by inner products
- Define the kernel function  $K$  by

$$K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$$

# An Example for $\phi(\cdot)$ and $K(\cdot, \cdot)$

- Suppose  $\phi(\cdot)$  is given as follows

$$\phi\left(\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}\right) = (1, \sqrt{2}x_1, \sqrt{2}x_2, x_1^2, x_2^2, \sqrt{2}x_1x_2)$$

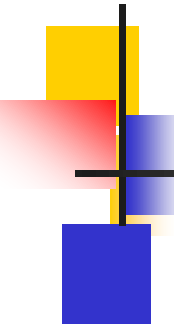
- An inner product in the feature space is

$$\langle \phi\left(\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}\right), \phi\left(\begin{bmatrix} y_1 \\ y_2 \end{bmatrix}\right) \rangle = (1 + x_1y_1 + x_2y_2)^2$$

- So, if we define the kernel function as follows, there is no need to carry out  $\phi(\cdot)$  explicitly

$$K(\mathbf{x}, \mathbf{y}) = (1 + x_1y_1 + x_2y_2)^2$$

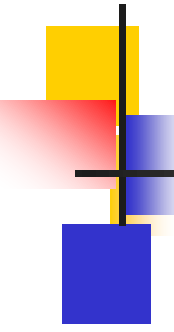
- This use of kernel function to avoid carrying out  $\phi(\cdot)$  explicitly is known as the **kernel trick**



# More on Kernel Functions

---

- Not all similarity measures can be used as kernel function, however
  - The kernel function needs to satisfy the **Mercer function**, i.e., the function is “positive-definite”
- This implies that
  - the  $n$  by  $n$  kernel matrix,
  - in which the  $(i,j)$ -th entry is the  $K(\mathbf{x}_i, \mathbf{x}_j)$ , is always positive definite
- This also means that optimization problem can be solved in polynomial time!



# Examples of Kernel Functions

---

- Polynomial kernel with degree  $d$

$$K(\mathbf{x}, \mathbf{y}) = (\mathbf{x}^T \mathbf{y} + 1)^d$$

- Radial basis function kernel with width  $\sigma$

$$K(\mathbf{x}, \mathbf{y}) = \exp(-\|\mathbf{x} - \mathbf{y}\|^2 / (2\sigma^2))$$

- Closely related to radial basis function neural networks
- The feature space is infinite-dimensional

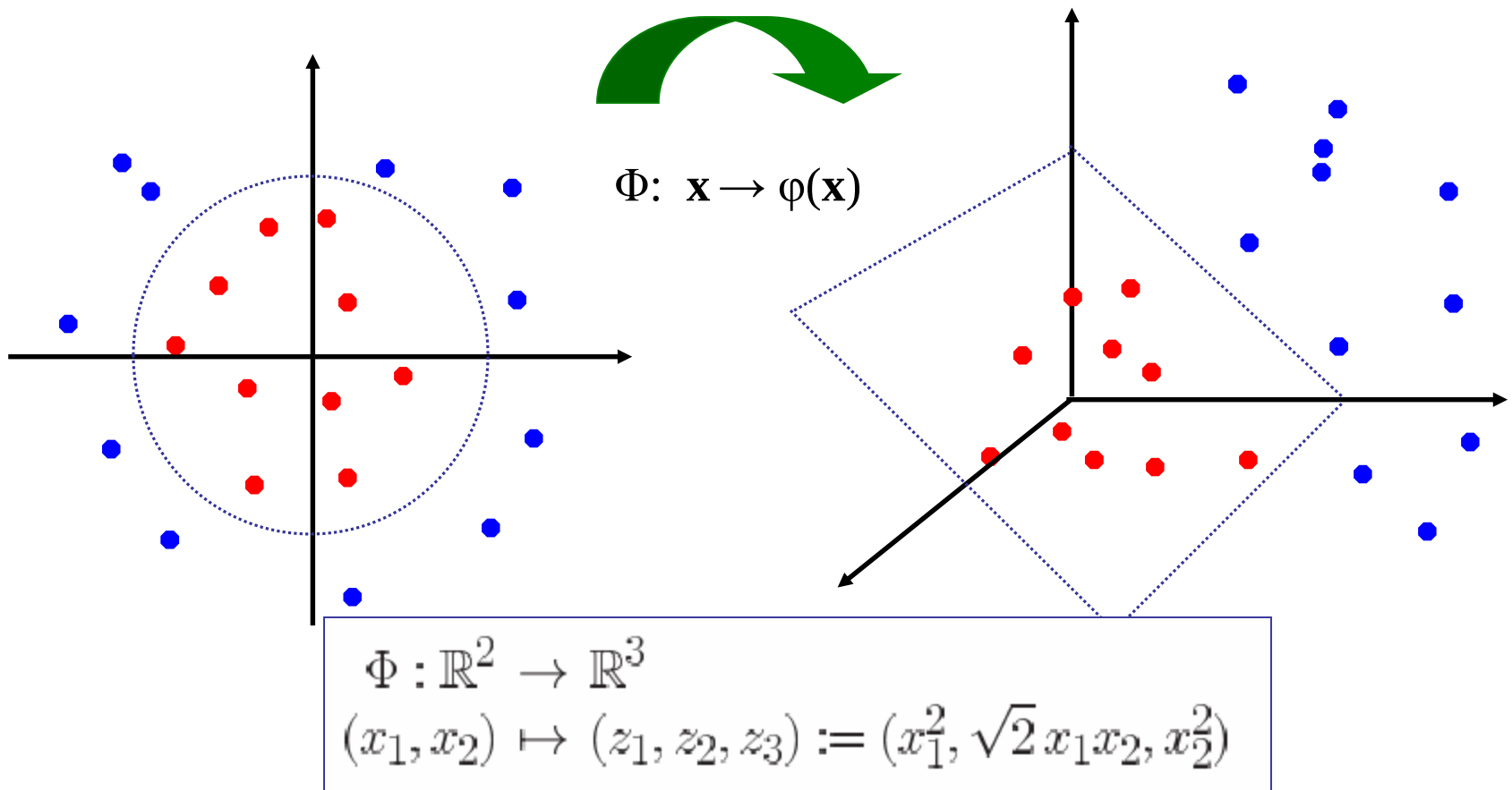
- Sigmoid with parameter  $\kappa$  and  $\theta$

$$K(\mathbf{x}, \mathbf{y}) = \tanh(\kappa \mathbf{x}^T \mathbf{y} + \theta)$$

- It does not satisfy the Mercer condition on all  $\kappa$  and  $\theta$

## Non-linear SVMs: Feature spaces

**General idea:** the original input space can always be mapped to some higher-dimensional feature space where the training set is separable:



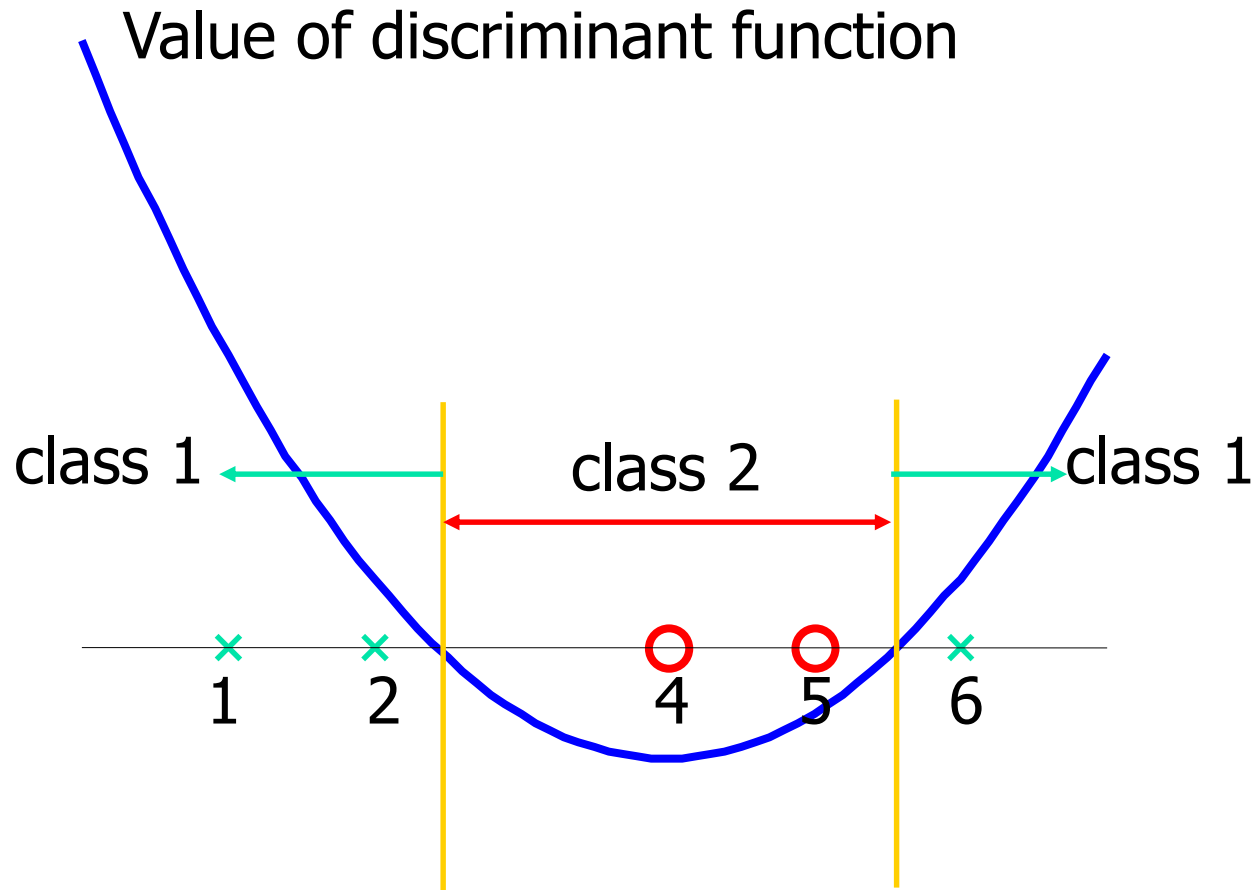


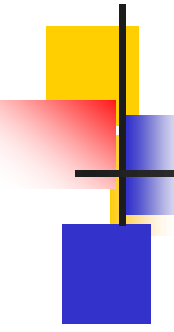
# Example

- Suppose we have 5 one-dimensional data points
  - $x_1=1, x_2=2, x_3=4, x_4=5, x_5=6$ , with 1, 2, 6 as class 1 and 4, 5 as class 2  $\Rightarrow y_1=1, y_2=1, y_3=-1, y_4=-1, y_5=1$
- We use the polynomial kernel of degree 2
  - $K(x,y) = (xy+1)^2$
  - $C$  is set to 100
- We first find  $\alpha_i$  ( $i=1, \dots, 5$ ) by

$$\begin{aligned} \max. \quad & \sum_{i=1}^5 \alpha_i - \frac{1}{2} \sum_{i=1}^5 \sum_{j=1}^5 \alpha_i \alpha_j y_i y_j (x_i x_j + 1)^2 \\ \text{subject to} \quad & 100 \geq \alpha_i \geq 0, \sum_{i=1}^5 \alpha_i y_i = 0 \end{aligned}$$

# Example

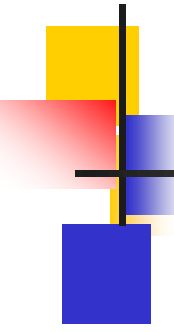




# Conclusion

---

- SVM is a useful alternative to neural networks
- Two key concepts of SVM: maximize the margin and the kernel trick
- Many SVM implementations are available on the web for you to try on your data set!



# Resources

---

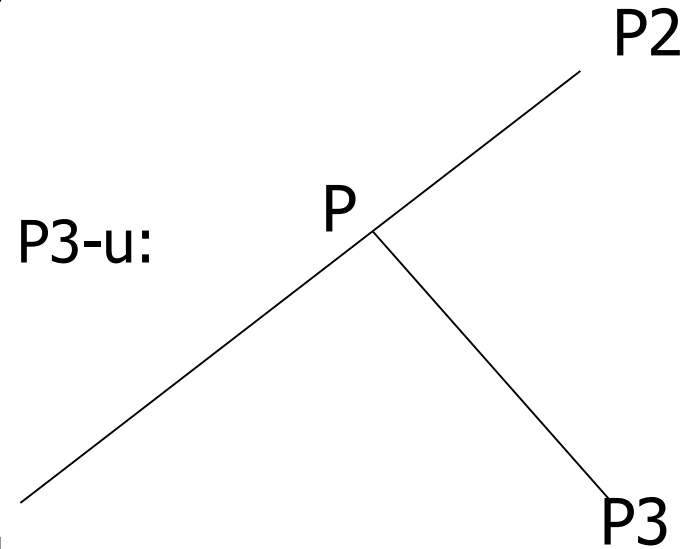
- <http://www.kernel-machines.org/>
- <http://www.support-vector.net/>
- <http://www.support-vector.net/icml-tutorial.pdf>
- <http://www.kernel-machines.org/papers/tutorial-nips.ps.gz>
- <http://www.clopinet.com/isabelle/Projects/SVM/applist.html>

# Appendix: Distance from a point to a line

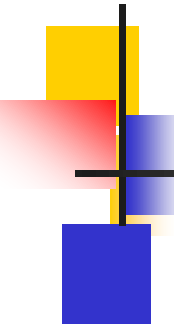
- Equation for the line: let  $u$  be a variable, then any point on the line can be described as:

$$\blacksquare \mathbf{P} = \mathbf{P1} + u (\mathbf{P2} - \mathbf{P1})$$

- Let the intersect point be  $u$ ,
- Then,  $u$  can be determined by:
  - The two vectors  $(P2-P1)$  is orthogonal to  $P3-u$ :
  - That is,
    - $(P3-P) \text{ dot } (P2-P1) = 0$
    - $P = P1 + u(P2-P1)$
  - $P1=(x1,y1), P2=(x2,y2), P3=(x3,y3)$



$$u = \frac{(x3 - x1)(x2 - x1) + (y3 - y1)(y2 - y1)}{\|p2 - p1\|^2}$$



# Distance and margin

---

$$u = \frac{(x_3 - x_1)(x_2 - x_1) + (y_3 - y_1)(y_2 - y_1)}{\|p_2 - p_1\|^2}$$

- $x = x_1 + u (x_2 - x_1)$   
 $y = y_1 + u (y_2 - y_1)$

- The distance therefore between the point **P3** and the line is the distance between  $P=(x,y)$  above and **P3**
- Thus,
  - $d = |(P_3 - P)|$